ePRF App

How The Code Works

A plain-English guide to app.js

Written for non-developers and beginners

Introduction

The ePRF app is an electronic Patient Report Form — it helps ambulance crews record everything that happened on a job, from the patient's symptoms, to the observations taken, medications given, and the final decision about what to do with them.

The whole app lives in two files: index.html (the layout and buttons you see on screen) and app.js (all the logic that makes it work). This guide walks through app.js section by section and explains what each part does in plain English.

You don't need to know how to code to read this. Wherever there's a technical word or concept, it will be explained.

> ■ *app.js is roughly 8,700 lines long, but it is broken into clear sections. Once you understand the pattern, each section follows the same logic.*

1 — The Very First Line: "use strict"

The very first line of app.js is:

```
"use strict";
```

This is a safety instruction to the browser. It tells JavaScript to be stricter about mistakes — for example, if you accidentally create a variable with a typo in its name, JavaScript will throw an error straight away rather than silently carrying on. Think of it like turning on spell-check. It helps catch bugs early.

2 — Shortcut Helpers

The next few lines create four small helper functions that are used throughout the rest of the file. They exist purely to save typing.

$ and $$

When you want to interact with something on screen — a button, an input box, a section — you need to find it first. The browser has a built-in way to do this, but it is quite verbose:

```
document.querySelector("#someId")
```

To save writing that every time, the code creates two shortcuts:

```
const $ = (selector, root = document) => root.querySelector(selector);
const $$ = (selector, root = document) => [...root.querySelectorAll(selector)];
```

$ finds one element on the page (the first match). $$ finds all matching elements and returns them as a list. The ? you'll see dotted throughout — like $("#someId")?.value — means "only try to read .value if the element actually exists; if not, just return undefined rather than crashing."

> ■ *Think of $ like searching for a specific person in a room. $$ is like finding everyone wearing the same colour.*

val()

```
const val = (id) => ($("#" + id)?.value || "").trim();
```

val("someId") reads whatever has been typed into a form field (like a text box or dropdown). It also trims off any accidental spaces at the start or end. If the field is empty or doesn't exist, it returns an empty string rather than crashing.

isChecked()

```
const isChecked = (id) => Boolean($("#" + id)?.checked);
```

isChecked("someId") returns true or false depending on whether a checkbox is ticked. Simple.

onsetTime() and onsetClockSuffix()

These two helpers read the onset time from the form. If the user selected "Other" from a dropdown, it reads the extra text box they typed in instead. onsetClockSuffix adds a formatted clock time in brackets if one was entered.

3 — Blood Pressure and Red Flags

rfSystolic()

Blood pressure is typed in as a combined reading like "120/80". rfSystolic() splits that text on the slash and returns just the first number (120) as a proper number. If there's no blood pressure entered, it returns null (meaning "nothing here").

evaluateRedFlags()

This function is a placeholder for future logic that would check whether any clinical red flags are present. For now it simply returns an empty list — but the rest of the code is already set up to use its results when that logic is added.

4 — The State Object

The state object is the memory of the app. Every time a user ticks a box, selects a chip, or adds an entry, that selection is saved into state. When it comes time to build the final report text, the code reads everything back out of state.

```
const state = {
mapMode: "site",
siteParts: new Set(),
ivEntries: [],
drugEntries: [],
...
};
```

There are two types of containers used inside state:

- Set — a list that automatically removes duplicates. Used for chip selections like character, associated symptoms, or injury types. Adding "Sharp" twice still only stores it once.
- Array ([ ]) — an ordered list where duplicates are allowed. Used for entries that can be repeated, like IV access attempts or medications given.

Each key in state corresponds to a specific part of the form:

- siteParts / radiationParts — body map selections for pain site and radiation
- character, associated, exacerbating, relieving — SOCRATES pain assessment chips
- ivEntries, drugEntries — IV access and medication entries
- seizureType, seizureFeatures — seizure assessment chip groups
- strokeFaceFindings, strokeArmFindings, etc. — stroke FAST assessment
- safeguardingConcerns, mcaAbilities — safeguarding and mental capacity
- ecgFindings, ecgLeads — ECG assessment chips
- clinicalChanges — clinical change log entries

■ *If you want to know what a user selected, look at state. It is the single source of truth for all adult form data.*

5 — Fixed Data Lists

CONVEY_TRANSFER

This is a list of pairs used in the conveyance checklist — each pair contains the "all good" statement and the "problem" statement for that item. For example:

```
["Consent to conveyance obtained", "Consent not obtained"]
```

Whichever one the user selects gets included in the handover text.

WORSENING_GENERIC and WORSENING_PC

These contain the worsening advice given to patients when they are left at home. WORSENING_GENERIC is a list of universal warning signs (like chest pain or seizure) that apply to any patient. WORSENING_PC is a larger object keyed by presenting complaint — if the patient's complaint was "Headache", the code can look that up and get the specific headache warning signs.

The redFlags key inside some entries (like Fall and Back pain) adds a special cauda equina warning that is shown separately.

6 — The OPTIONS Object

OPTIONS is a large configuration object that contains the lists of items to show in every chip grid and dropdown throughout the form. Instead of hard-coding button labels scattered all over the HTML, they are all stored here in one place.

Examples of what is inside:

- OPTIONS.pain.character — the list of pain character words: Sharp, Dull, Aching, Burning, etc.
- OPTIONS.pain.exacerbating — things that make the pain worse: Movement, Inspiration, After eating, etc.
- OPTIONS.falls.symptoms — symptoms at time of fall: Dizziness, Palpitations, Tripped, etc.
- OPTIONS.seizure — types of seizure, features, precipitants, post-ictal features
- OPTIONS.stroke — stroke findings for FAST assessment

When the app starts up, functions like buildButtonGrid() read from OPTIONS to automatically create all the buttons on screen. That means if you ever need to add or remove a chip option, you only need to change it in OPTIONS — everywhere that chip appears will update automatically.

7 — Populate and Build Functions

Before the user can interact with the form, all the interactive chip grids and dropdowns need to be created. These functions run once when the app loads and fill in the parts of the page that are too repetitive to write manually in HTML.

buildButtonGrid()

This is the workhorse function for creating chip buttons. You pass it the ID of a container element, a list of options, and some configuration, and it creates a grid of buttons inside that container. Every single chip grid in the app is built this way.

```
buildButtonGrid("characterGrid", OPTIONS.pain.character, "pain", "character",
"someHiddenInput")
```

populateSelects()

This fills in the dropdown menus (select elements) throughout the form. Rather than putting lots of <option> tags in the HTML, this function runs on startup and adds the options from data arrays. Keeps the HTML clean.

populateSymptomSearchOptions()

Fills in the symptom search tool so users can quickly look up worsening advice for any presenting complaint.

buildStrokeCard() and similar card builders

Some assessment sections are complex enough to need their own dedicated builder function. buildStrokeCard() creates the full stroke FAST assessment grid with its chip rows. These functions run when the relevant section is first shown.

8 — The init() Function

init() is the starting point of the whole app. It is called once as soon as the page loads, and it sets everything in motion.

Here is what init() does, in order:

- Calls populateSelects() to fill in all dropdowns
- Calls buildButtonGrid() for every chip grid on the adult form
- Builds the GCS calculator for the primary survey and head injury sections
- Builds the stroke assessment card
- Calls bindEvents() to attach all the click, change, and input listeners
- Sets up the dashboard cards
- Applies the saved theme (dark/light mode)

■ *If something does not appear on screen when the app loads, init() is the first place to check.*

9 — The Dashboard and Feature Switching

showDashboard() and showFeature()

The app has a dashboard with feature cards (ePRF, Obs Recorder, NEWS2, Drug Finder, Resp Counter). showDashboard() hides everything except those cards. showFeature(featureName) hides the dashboard and shows whichever feature was clicked.

Features are lazy-loaded — the obs recorder and NEWS2 tool are only fully initialised the first time they are shown. After that, the init flag (_obsRecInited, _news2Inited) prevents them from being set up twice.

initDashboard()

Attaches click listeners to each dashboard card, plus the back button and the BNF drug search form. The BNF search opens the NICE BNF website in a new browser tab with the search term pre-filled.

10 — Entry Managers

Several parts of the form allow the user to add multiple entries — for example, multiple IV access attempts or multiple medications. The makeEntryManager factory function handles all of that in a reusable way.

makeEntryManager()

```
const { render, remove } = makeEntryManager(stateKey, containerId, formatFn, removeAttr, stateObj)
```

You call makeEntryManager once for each type of entry, passing:

- stateKey — the name of the array in state to store entries (e.g. "ivEntries")
- containerId — the ID of the div where rendered entries should appear
- formatFn — a function that turns one entry object into a readable string
- removeAttr — the data attribute name used on the remove button
- stateObj — which state object to use (defaults to state for adult, paedsState for paediatric)

makeEntryManager returns two functions: render() which redraws all entries from the array, and remove(index) which deletes one entry and redraws.

addIvEntry()

Reads the access type, site, gauge, outcome, flush, and fluids fields from the form. If the required fields (type and site) are filled in, it pushes a new entry object into the state array and calls render() to show it. It then clears the form fields ready for the next entry.

addDrugEntry() and renderDrugEntries()

Works the same way as IV entries but for medications. renderDrugEntries() also adds a repeat button (■) next to each entry. Clicking repeat pre-fills the drug form with that entry's details and updates the time to now, making it quick to give a second dose.

addChangeEntry() and renderChangeEntries()

Records a clinical change — a timestamped note that something changed during the call. Used in the clinical changes section of the form.

11 — bindEvents()

bindEvents() is where all the interactive behaviour of the adult form is wired up. It runs once during init() and attaches event listeners to every button, input, dropdown, and chip group that needs to do something when clicked or changed.

An event listener is just a piece of code that says: "when this thing happens, run this function." For example:

```
$("#addVaButton")?.addEventListener("click", () => addIvEntry());
```

That line means: find the button with ID addVaButton, and when it is clicked, call addIvEntry().

Examples of what bindEvents sets up:

- Add IV access button → calls addIvEntry()
- Add medication button → calls addDrugEntry()
- VA type dropdown → shows or hides gauge/site chip groups depending on whether it's an IV or IO
- VA outcome dropdown → shows or hides the flush chip group
- ABCDE chip clicks → toggles the chip between normal/abnormal state
- GCS buttons → updates the GCS total score
- Pain score buttons → saves the selected severity
- Copy output button → copies the report text to the clipboard
- Reset button → clears the entire form

■ *If a button does nothing when you click it, the event listener for it is either missing from bindEvents(), or there is an error in the function it calls.*

12 — Text Builders

The whole point of the app is to generate a structured text report at the end. The text builder functions read everything from state and the form, and piece together the final paragraphs of the ePRF.

buildOutput()

This is the main text builder. It reads all the form data and produces the complete report in whichever format was selected — Standard, SBAR, ATMIST, or Leave at Home.

Standard format is the default: it produces flowing paragraphs for each section (patient details, presenting complaint, SOCRATES, observations, interventions, etc.).

SBAR (Situation, Background, Assessment, Recommendation) is a structured handover format used when handing over to a hospital team.

ATMIST (Age, Time, Mechanism, Injuries, Signs, Treatment) is used for trauma handovers.

Leave at Home produces the SBAR-style text for when the patient is being left at home.

buildLahSbarText()

Specifically builds the Leave at Home version of the report, including the worsening advice section and what the patient was told to do if things get worse.

buildObsText()

Reads the observations recorded in the obs sets (obs recorder) and formats them as text. Calculates NEWS2 scores for each set and flags any abnormals.

buildHeadInjuryText() and buildConveyanceText()

Focused text builders for specific sections — head injury assessment and conveyance decision. Each reads the relevant state and form fields and returns a formatted string.

getNiceCTCriteria()

Checks whether any of the NICE criteria for CT scanning are met (based on the head injury section). Returns a formatted string listing which criteria apply. Uses the GCS values from the head injury GCS calculator.

13 — NEWS2 Scoring

NEWS2 is the National Early Warning Score 2 — a clinical scoring system used to identify patients at risk of deterioration. It gives a score for each vital sign, and the total determines how urgently the patient needs review.

newsScore()

This function takes the six vital sign values and consciousness level and returns a total NEWS2 score plus a breakdown of the individual scores. It uses the official NHS NEWS2 thresholds:

- Respiratory rate — scored 0-3 based on how far outside normal range
- SpO2 — two different scales (Scale 1 for most patients, Scale 2 for COPD/hypercapnic patients)
- Supplemental oxygen — 2 points if the patient is on oxygen
- Systolic BP — scored 0-3
- Heart rate — scored 0-3
- Temperature — scored 0-3
- AVPU consciousness — 3 points for anything other than Alert

initNews2() and updateNews2()

initNews2() sets up the standalone NEWS2 tool (accessible from the dashboard). It only runs once. updateNews2() is called whenever any vital sign changes in the obs recorder — it recalculates the NEWS2 score for that obs set and updates the display.

NEWS2_GUIDANCE

A lookup object that maps NEWS2 score ranges to the recommended clinical response — from "routine monitoring" at score 0, up to "emergency assessment" at score 7 or above.

14 — Obs Recorder and Obs Sets

The obs recorder allows the crew to record multiple sets of observations over time — for example, obs every five minutes during a long transfer. Each obs set is a self-contained block with a full set of vital signs fields.

initObsRecorder()

Runs the first time the Obs Recorder feature is opened. Creates the first obs set automatically, then wires up the "Add Obs Set" button to create more.

createObsSet()

Builds one complete obs set — a div containing all the vital signs input fields, chip buttons for AVPU/consciousness, remove button, and event listeners. Each obs set is self-contained; they don't interfere with each other.

createObsSet() also sets up:

- A remove button that deletes that obs set and renumbers the remaining ones
- Chip click listeners for AVPU, on-oxygen, and similar binary choices within the set
- Input listeners that trigger NEWS2 recalculation whenever a value is typed

updateObsSetNumbers()

After adding or removing obs sets, this function renumbers the headers — "Obs Set 1", "Obs Set 2", etc.

15 — GCS Calculator

GCS stands for Glasgow Coma Scale — it is a way of scoring a patient's level of consciousness by looking at their eye opening, verbal response, and motor response. It produces a score out of 15.

buildGcsCalcHTML()

Creates the HTML for a GCS calculator widget. It accepts a prefix parameter so that the same function can build multiple GCS calculators on the page — one for the primary survey, one for the neuro section, one for head injury — without them conflicting.

```
buildGcsCalcHTML("headGcs") // builds a calculator with IDs starting headGcs...
```

The calculator uses button-chips for each level of Eye (1-4), Verbal (1-5), and Motor (1-6). When a button is clicked, its value is saved in a data attribute on the button, and the total is recalculated.

updateGcsTally()

Called whenever a GCS button is clicked. Reads the selected value for each of Eye, Verbal, and Motor, adds them up, and updates the total display. For the primary survey and neuro calculators, it also writes the total into the main GCS score field so it feeds into the output text.

The head injury calculator deliberately does not write to the main GCS field — it has its own display and is used separately for the NICE CT criteria check.

How the selected value is stored

Rather than using a hidden input, the GCS calculator stores the selected value in a data attribute on the button element itself:

```
button.dataset.gcsSelected = "3"
```

When reading the value back, the code uses:

```
parseInt($("#headGcsEye")?.dataset.gcsSelected || "", 10) || 0
```

This is called the sentinel pattern — if no button has been selected, the dataset value is empty and the code safely returns 0 instead of crashing.

16 — Conveyance Section

Conveyance refers to transporting the patient — to hospital, to a GP, to a walk-in centre, or leaving them at home. The conveyance section of the app handles recording the decision and the handover details.

buildConveyDestination()

Creates the destination chip group — hospital, GP, walk-in centre, etc. Accepts a prefix so it can serve both the adult form ("convey") and the paeds form ("pConvey") without duplication.

toggleConveyChip()

Handles clicking a conveyance checklist chip (like "Consent obtained" or "Pre-alert given"). Each pair in CONVEY_TRANSFER has a positive and a negative chip. Selecting one deselects the other.

buildConveyanceText()

Produces the handover text for the conveyance section — which destination was chosen, all the checklist items, and any additional notes. This feeds into the SBAR Recommendation section.

17 — Paediatric Mode

When the paeds mode toggle is switched on, the app shows a different, age-appropriate assessment form. The paeds form has its own state object, its own set of chip grids, and its own text builder.

paedsState

A separate state object that mirrors the adult state but contains only the fields relevant to paeds assessments:

- pIvEntries — paediatric vascular access entries
- pDrugEntries — paediatric medication entries
- pAirwayInterventions, pWoundInterventions, pPositioningInterventions — treatment selections
- pReferrals — referral selections

Having a separate paedsState means that switching between adult and paeds mode does not overwrite each other's data.

initPaeds()

Runs the first time paeds mode is activated. Builds all the paeds-specific chip grids (ABCDENT assessment, PAT triangle, FLACC pain scale, safeguarding grid, treatment sections) and attaches their event listeners.

A guard flag (_paedsInited) ensures this only runs once — if the user toggles in and out of paeds mode, the form is not rebuilt each time.

PAT Triangle

The Paediatric Assessment Triangle (PAT) is a rapid visual assessment using three areas: Appearance, Work of breathing, and Circulation to skin. Each area has chip buttons that toggle between normal and abnormal states. This gives a quick first impression of how sick the child is.

FLACC Pain Scale

FLACC is a pain assessment tool for children who cannot verbally communicate their pain. It scores five areas (Face, Legs, Activity, Cry, Consolability) on a 0-2 scale. updateFlaccScore() sums those scores and displays the total with a label: mild, moderate, or severe.

18 — Respiratory Rate Counter

The respiratory rate counter helps accurately count a patient's breathing rate. Rather than relying on estimation, the crew starts a timer and taps a button once for each breath. The app calculates the rate per minute automatically.

respCounter object

A small object that holds the counter's current state:

- duration — the counting window in seconds (15, 30, or 60)
- running — whether the counter is currently active
- startTime / endTime — timestamps for calculating elapsed time
- count — how many taps have been recorded
- timerId — the interval timer handle (used to stop the timer later)

startRespCounter()

Starts the count. Sets the start and end timestamps, disables the duration selector so it cannot be changed mid-count, and starts a timer that fires every 250ms to update the display.

recordRespTap()

Called each time the tap button is pressed. Increments the count by 1 and adds a brief pulse animation to the tap button as visual feedback.

finishRespCounter()

Called automatically when the counting window expires. Stops the timer, calculates the final rate (taps divided by duration, multiplied by 60), and shows the result card. If the rate is below 12 or above 20 (outside the normal range), the result card is highlighted in a warning colour.

resetRespCounter()

Resets everything back to zero. Optionally clears the result card as well.

19 — Dark / Light Mode Theme

The theme code is wrapped in an immediately-invoked function — a block of code that runs itself as soon as the page loads. It:

- Reads the saved theme preference from localStorage (the browser's built-in storage that persists between sessions)
- Applies that theme by setting a data-theme attribute on the root html element
- Updates the toggle button label and the browser tab colour
- Attaches a click listener to the theme toggle button so it switches and saves the preference

Using a data-theme attribute on the root element means all the CSS colour variables can switch at once — no JavaScript needed to change individual colours.

20 — How It All Fits Together

Here is a simple walk-through of what happens from the moment the page loads to the moment a report is generated:

1. The browser loads index.html. The HTML file sets up all the sections, cards, and input fields that will be visible.
2. app.js loads. "use strict" is set. The helper functions ($, $$, val, isChecked) are defined.
3. The state object is created — empty, ready to receive data.
4. All the OPTIONS, WORSENING, and CONVEY_TRANSFER data objects are defined.
5. init() runs. Dropdowns are populated. Chip grids are built. GCS calculators are injected. bindEvents() wires up all the interactivity.
6. The crew uses the form. Every selection, tap, and typed value updates state (or paedsState).
7. When "Copy" is pressed, buildOutput() reads everything from state and produces the complete text report.

That's the whole app — seven steps from page load to finished report.

The key insight is separation of concerns: the HTML defines what you see, OPTIONS defines what the choices are, state holds what was selected, and the text builders turn state into the final report. Each part has one clear job.

> ■ *The best way to understand a new section of code is to ask: "what does it read from?", "what does it write to?", and "when does it run?" Almost everything in app.js can be understood from those three questions.*